

Playing Billiards with the Unitree G1

Sullivan Hart

I. OVERVIEW

This project simulates the Unitree G1 humanoid robot playing pool in MuJoCo. The robot stands at a pool table, uses a cue stick to break the rack, then teleports to follow-up shot positions and takes real-contact shots until no makeable shots remain. The simulator uses the absolute state of all objects to plan positions for the robot. All ball outcomes are determined by MuJoCo physics contacts.

A. Motivation

I chose to make a billiards robot because it builds on in-class topics, has real-world application, and allows me to incorporate an extracurricular interest. The project also had a very adjustable scope, and this course is my first experience with robotics; I wanted to do something challenging but not impossible, and being able to add and remove elements to the project scope helped me get an appropriate amount of complexity. This project could have been much simpler (no standing controller, non-humanoid robot, non-realistic physics, etc.), or it could have been much more complex (computer vision for shot planning, competition between multiple robots, building a well-suited walking controller, etc.). Regardless, I was excited to do a project where the robot does something.

B. Challenges

The primary challenge was getting the robot to remain upright while striking. The cue stroke exerts a lateral force on the arm, and the same controller that must resist this disturbance is the one computing leg torques. Getting reliable ball-cue contact through MuJoCo's contact geometry (collision group and mask settings, solver parameters) also required significant iteration. Walking was also a significant challenge; the walking controller that I attempted to integrate did not get to a state where it was able to walk to the exact point it needed to make the shot. Controlling the arm was

similar, it was not able to get enough precision to be included in the final version of the simulation. Each added component (walking controller, standing controller, shot planner, etc.) introduced more error and uncertainty to the behavior of the robot.

II. IMPLEMENTATION

A. Scene and Physics Setup

The pool table and all balls are built from MuJoCo geometries (boxes, cylinders, spheres) rather than meshes, which gives direct control over collision properties. The table cloth uses a friction coefficient of 0.16 for rolling resistance; cushions use 0.025 with a soft contact solver (`solref="0.0018 0.18"`) to produce realistic rebound. Each of the 15 object balls and the cue ball is a free body (sphere, radius 28.575 mm, mass 170 g). Rolling resistance beyond MuJoCo's damping is applied at each simulation step as a small velocity-dependent force, tuned to a resistance coefficient of 0.012.

Six pockets (four corners, two sides) are represented by cylinder geoms with `ctype=0/conaffinity=0`, so they are visible but do not collide. Pocketing is detected each step by comparing ball center XY against hardcoded pocket radii and pocket-mouth checks; pocketed balls are frozen at a sub-table position to remove them from play.

The simulation runs at $\Delta t = 0.002$ s (500 Hz). The standing controller is invoked every tenth step (50 Hz control decimation). The scene XML includes the Unitree G1 robot, the cue body, the billiards table, and all ball bodies.

B. Cue Stick Attachment

The cue stick is attached to the robot as a child body in the right wrist/hand chain. A 1-DOF slide joint (`cue_slide_joint`, axis along the local x -direction, range $[0, 0.5]$ m) lets the cue extend forward from the grip. The cue body

(right_cue) has a 1.4 m shaft geom with collision disabled and a 3 cm tip geom (cue_tip) with `contype=1/conaffinity=1` that can contact the cue ball. A site (cue_tip_site) at the tip is used for alignment geometry.

The slide joint is driven kinematically: each simulation step sets `data.qpos[cue_slide_qadr]` and `data.qvel[cue_slide_dadr]` directly from a time-parameterized profile. For the break shot, the cue extends 0.35 m over 0.14 s, holds briefly, then retracts.

C. Balance Controller

The standing controller (StandingCtrl) runs at 50 Hz and produces joint torques for the 27 actuated robot DOF.

1) *Leg Torques via QP*: Leg torques are computed using a Lagrangian dynamics decomposition. A *whole-body QP* solves for the 12-component ground reaction wrench (3-force + 3-moment at each foot). The QP is formulated once with CVXPY `cp.Parameter` objects and is cached for re-solving on each call; this avoids the per-call CVXPY compilation overhead that was causing issues during long simulation runs.

The QP objective minimizes deviation from a target floating-base acceleration while regularizing the contact forces and limiting the center of pressure:

$$\begin{aligned} \min_{\mathbf{w}} \quad J(\mathbf{w}) = & \|H_b \mathbf{w} + h_b\|_{W_b}^2 \\ & + \|f_{z,L} + f_{z,R} - F_z^*\|_{W_t}^2 \\ & + W_r \|\mathbf{w}\|^2. \end{aligned} \quad (1)$$

subject to relevant constraints (friction cone, CoP, and torsion) at each foot.

Because the QP runs at the intended standing pose, a gravity-correction term is added to compensate for the difference between the intended and actual robot configuration:

$$\boldsymbol{\tau}_{\text{leg}} = \boldsymbol{\tau}_{\text{QP}} + \mathbf{G}_{\text{current}}[6:] - \mathbf{G}_{\text{standing}}[6:] \quad (2)$$

where $\mathbf{G}$ is the pure gravity force computed by `mj_rne` with zero velocity and acceleration.

A PID loop further drives the estimated center of mass toward the target standing CoM position, distributing corrective forces to the ankle and hip pitch joints. Integral terms on the base XY position prevent slow lateral drift.

2) *Arm Torques*: The arm and waist joints are controlled by a pure PD law targeting the desired shot pose. To prevent arm sag under gravity, an additional feedforward term is computed at each step:

$$\boldsymbol{\tau}_{\text{arm}} = K_p(q^* - q) - K_d \dot{q} + \mathbf{G}_{\text{arm}}(q^*, q_{\text{legs}}) \quad (3)$$

The feedforward $\mathbf{G}_{\text{arm}}$ is extracted from an auxiliary `MjData` object that is set to the *desired* arm pose (not the actual) while leaving leg configuration at its current values. The resulting `qfrcbias` vector at the arm DOF addresses gives the exact gravity torque required to hold the arm at the target, so the PD controller only needs to correct tracking error rather than fight static gravitational load.

The final implementation keeps this pitch compensation disabled and relies on the fixed arm target and gravity feedforward for cue stability.

D. Shot Planning

Shot planning uses the ghost-ball model. For each live object ball and each pocket, the *ghost-ball position* is placed one ball diameter behind the object ball along the pocket line. The required cue-ball direction is the vector from the current cue-ball position to the ghost-ball. A cut-alignment score measures the dot product of this vector with the object-to-pocket direction:

$$\text{score} = 3.0 \alpha_{\text{cut}} - 0.25 d_{\text{cue}} - 0.20 d_{\text{obj}} + 0.15 I_{\text{side}} \quad (4)$$

where I_{side} is one for side pockets and zero otherwise.

Conservative shots with $\alpha_{\text{cut}} \approx 0.70$ or higher are preferred because thin shots tend to fail in simulation. A segment-clear check rejects shots where another ball lies within 2.15 ball radii of either the cue-to-ghost or object-to-pocket line. A relaxed planner keeps a reduced cue-path clearance check and treats object-to-pocket blockers as a soft penalty when no conservative shot exists.

The robot pose for a shot is computed from the cue-ball position and the required shot direction:

$$\mathbf{x}_{\text{robot}} = \mathbf{x}_{\text{cue}} - R(\psi) \cdot \mathbf{d}_{\text{local}} \quad (5)$$

where $\mathbf{d}_{\text{local}} = [1.50, -0.116]^T$ m is the fixed local offset from robot base to cue-ball contact point at the break pose, and ψ is the shot yaw.

E. Multi-Shot Execution

Every shot uses the same `run_physical_stroke` function with the live standing controller. For the break, the robot starts from its initial stable standing pose. For follow-up shots the robot is first teleported to the computed shot pose, then the same stroke is run.

1) *Teleport Reset*: After each shot, the robot is teleported to the computed shot pose by loading the intended standing qpos into all robot DOF and writing the target XY and yaw into the free-joint slots. This full state reset prevents drift accumulation: the standing controller starts each shot from a known-good equilibrium rather than trying to recover from the previous shot's residual motion.

2) *Cue Alignment*: Before the cue slide begins, the robot's position is adjusted by up to 0.10 m to correct residual geometric offset between the cue tip site and the cue-ball center.

III. RESULTS

The system successfully executes multi-shot sequences without the robot falling. Shot 1 (the break) consistently strikes the rack and scatters the balls. Follow-up shots are planned from the actual post-break table state. The robot maintains mostly-stable upright posture throughout all shots.

Pocketing outcomes depend entirely on the MuJoCo physics and the quality of the selected shot. Thin cuts are avoided by the conservative planner, with a relaxed fallback used to continue making real target-ball contacts when no high-probability shot is available. Typical runs reliably pocket the break ball and at least one planned follow-up; later shots often make real target contact but are less reliable at pocketing the called ball.

IV. FUTURE WORK

- **Walking between shots.** The current implementation teleports the robot. A natural extension is to use a walking controller to walk around the table and reach the shot position, making the demonstration more physically realistic.
- **Cue-ball spin.** The current stroke applies no sidespin. Adding planned english (side english, draw, follow) would make the shot planner more powerful by allowing position play after each shot.

- **Multi-ball planning.** The planner evaluates shots independently. A tree search over two or three shots could select shots that leave the cue ball in a good position for subsequent shots, mimicking non-trivial pool strategy.
- **Two-robot competition.** The scene and planner are not specific to a single robot instance. A second G1 robot could be placed on the opposite side of the table, alternating turns like a real game.
- **Vision-based state estimation.** All current planning uses absolute MuJoCo state. Replacing this with the G1's onboard camera or an aerial camera and a ball-detection pipeline would make the system applicable to a real robot.